4-DAY HANDS-ON WORKSHOP PYTHON & SQL

About Me

Prakash Madariya

BE in **Computer Science Eng.**

Around **9 Years** of IT experience as **Full Stack Developer**

Currently working at **Virtusa** as **Tech Lead Engineer**

Mentored over **1000+ students**

WORKSHOP OBJECTIVE

- Build strong Python fundamentals
- Improve logical thinking and problem solving skills

Because

Companies don't hire based on theory

✓They test problem solving

✓They test logic

✓They test debugging

✓They test database understanding

WHAT YOU WILL BUILD

By Day 4:

- ✓ Connect Python with database
- ✓ Build mini backend system
- ✓ Write optimized queries
- ✓ Think like IT professional

WHERE PYTHON IS USED

- Backend development
  - **Roles:** Python Developer, Backend Engineer, API Developer, Django/Flask Developer
- Automation & scripting
  - **Roles:** Automation Engineer, Python Scripting Engineer, Automation Tester, Software Development Engineer in Test (SDET)
- Data engineering & AI
  - **Roles:** Data Engineer, Data Scientist, Machine Learning Engineer, Data Analyst
- DevOps & Cloud integration
  - **Roles:** DevOps Engineer, Cloud Engineer, Site Reliability Engineer (SRE), Infrastructure as Code (IaC) Engineer

DAY 1 – PYTHON FUNDAMENTALS AND LOGIC BUILDING

HOW PYTHON EXECUTES CODE

- Lexer convert the source code in tokens called tokenizer.

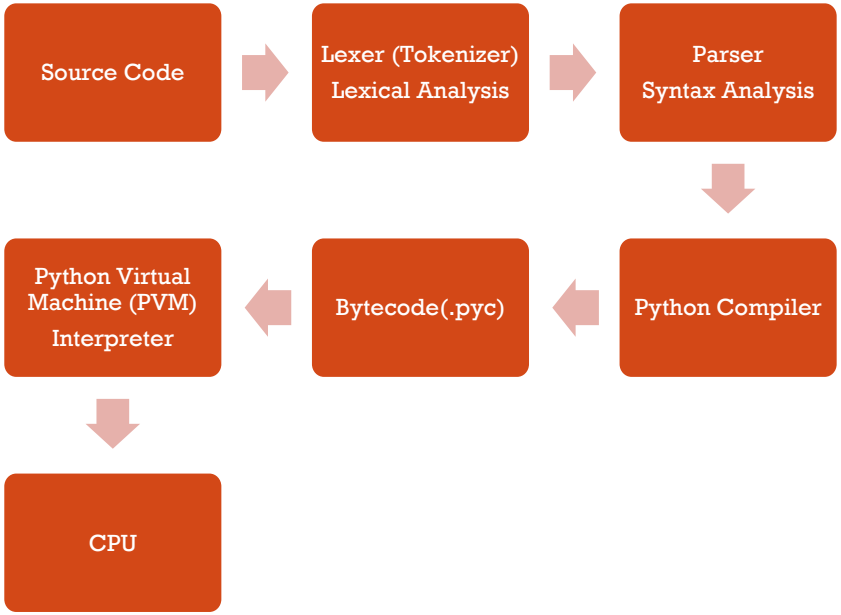
```
KEYWORD(if)
NAME(x)
OP(>)
NUMBER(10)
OP(:)
NEWLINE
INDENT
NAME(print)
LPAREN
STRING("Hi")
RPAREN
NEWLINE
DEDENT
```

- Parser checks whether tokens follow grammar rules.

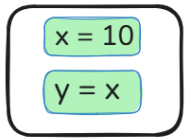
  If tokens don't match grammar → parser fails → SyntaxError.

  Colon missing → Parser cannot match grammar → SyntaxError.

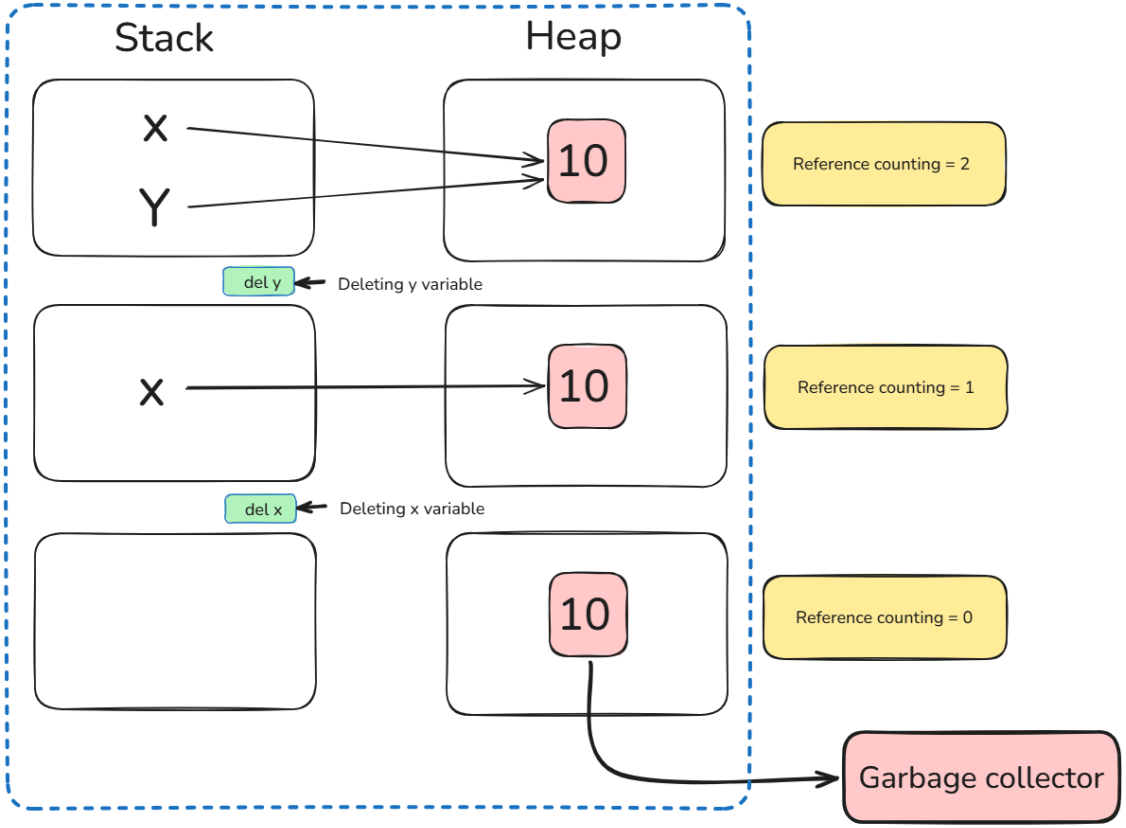
- The interpreter runs the program instruction by instruction and stops if a runtime error happens.



MEMORY MANAGEMENT AND GARBAGE COLLECTOR IN PYTHON



- Stack stores variable names (x, y)
- Heap stores actual object (10)
- y = x increases reference count
- del y decreases reference count
- del x makes reference count = 0
- Object becomes eligible for garbage collection



MODULE VS PACKAGE VS LIBRARY

Module

math_util.py

```
def add(a, b):
    return a + b
```

main.py

```
import math_util

print(math_utils.add(2, 3))
```

Package

```
├── my_project/
├── main.py
├── utils/
│   ├── __init__.py
│   ├── math_util.py
│   ├── string_util.py
```

main.py

```
from utils.math_utils import add

print(add(2, 3))
```

Library

```
student_utils/
│
├── src/
│   └── student_utils/
│       ├── __init__.py
│       ├── grading.py
│       └── validation.py
│
├── tests/
│
├── pyproject.toml
├── README.md
└── LICENSE
```

Metadata

Build system

Dependencies

Configuration

> pip install build
> python -m build

eg. pandas, numpy

CORE DATA TYPES (WITH USE CASES)

- int – IDs, counters
- float – prices, metrics
- str – user input, APIs
- list, tuple, set – collections of records
- dict – structured data (key-value)

MUTABLE VS IMMUTABLE DATA TYPES

Mutable: list, dict, set

Mutable → Can change after creation

- ✓ List is mutable because we modified it without creating a new object.
- ✓ Dictionary values can be changed.
- ✓ Set can be modified.

Immutable: int, str, tuple

Immutable → Cannot change after creation

- ✓ New int object is created when value changes.
- ✓ Strings cannot be modified directly.
- ✓ Tuple elements cannot be changed.

CONDITIONAL LOGIC IN REAL PROJECTS

Access control

- ✓ Authentication → Who are you?
- ✓ Authorization → What can you do?
- ✓ Role-Based Access Control (RBAC)

Validation checks

- ✓ Input validation (form validation)
- ✓ Business validation
- ✓ Cross-field validation(relationship between one field to another field)
- ✓ Database constraint validation

Feature toggles

- ✓ Gradual rollouts (increasing hike percentage)
- ✓ Emergency rollback
- ✓ Beta features

Business rules

Core domain logic that defines **how the system behaves**.

Example:

IF age > 60

AND pre-existing disease = True

AND coverage > 10L

THEN require medical test

LOOPS IN PRODUCTION CODE

Processing records

- ✓ Monthly salary processing
- ✓ EMI auto-deduction
- ✓ Subscription renewal
- ✓ Invoice generation

Iterating over API responses

- ✓ Fetch AWS logs
- ✓ Sync payment gateway data
- ✓ Collect analytics data

Reading files

- ✓ Upload bulk employee data
- ✓ Import banking transactions
- ✓ Read log files
- ✓ ETL pipelines
- ✓ AI dataset preprocessing

Handling batch operations

- ✓ Payroll processing
- ✓ Sending 10,000 emails
- ✓ Updating product prices
- ✓ Generating reports
- ✓ ML training data processing

COMMON BEGINNER MISTAKES

- No Input Validation
- Writing Everything in One Function
- Not Using Logging Properly
- Not Writing Tests
- Ignoring Performance
- Ignoring error messages
- Hardcoded values
- Copy-Paste Programming
- Ignoring Security Basics eg.(CSRF, Input sanitization)

DEBUGGING MINDSET

- Read the error message carefully
- Identify line number (e.g. File "app.py", line 45)
- Understand expected vs actual behavior
- Fix root cause, not symptom
- Reduce the Problem Scope - Simplify.

"Good developers write code. Great engineers debug efficiently."

WHAT COMPANIES LOOK FOR

- Clean, readable code
- Modular structure
- Edge case handling
- Proper naming conventions
- Debugging skills

HOW TO IMPROVE CODING SKILLS

- Solve 1 problem daily
- Practice on LeetCode / HackerRank
- Build small projects
- Review and refactor code

MINI CODE CHALLENGE

Student Ranking & Grading System

Problem Statement

- Create a program that:
- Takes number of students as input.
- For each student, take:
  - name
  - age
  - marks
- Store data in a list of dictionaries.
- Sort students by marks using lambda.
- Print ranked list using enumerate().
- Use match statement to assign grade:
  - 90+ → A
  - 75+ → B
  - 50+ → C
  - Otherwise → Fail

DAY 1 RECAP

- Python fundamentals and basics
- Data types and their built-in methods
- Conditional logic & loops
- Problem solving approach

ASSIGNMENT

- Build Student Grade Calculator
- Take Student name and input for 5 subjects
- Calculate average
- Assign grade based on marks

Features:
- Takes student name
- Takes marks for 5 subjects
- Validates marks (0–100)
- Calculates total and average
- Checks individual subject fail condition
- Assigns grade using match
- Displays formatted result

CLOSING NOTE

- Don’t aim to finish the workshop.
- Aim to think like a developer.
- See you on Day 2 – Functions & File Handling!